piKL – KL-regularized equilibrium search

Scope. This document specifies the search of the piKL agent: the KL-regularized equilibrium search of `src/entente/search/` and the agent that runs it over the behavioral-cloning network. It is written from A. P. Jacob, D. J. Wu, G. Farina, A. Lerer, H. Hu, A. Bakhtin, J. Andreas, N. Brown, *Modeling Strong and Human-Like Gameplay with KL-Regularized Search*, ICML 2022, arXiv:2112.07544: the update rule, the guarantees and the hyperparameters of Secs. 3–4 and Sec. 7 are restated from that paper in this project's notation, and everything else – the module boundaries, the candidate and utility constructions as implemented, the validation plan and the configuration – is my own. The modules of Sec. 9 implement what Secs. 2–7 define.

1 Why anchored search

Behavioral cloning predicts human orders well but plays weakly; unregularized equilibrium search on a 1-ply subgame – hedge or regret matching over sampled candidates – plays strongly but predicts human orders poorly, and play that differs markedly from human play is hard for humans to coordinate with and to understand. Jacob et al. reconcile the two by making each player optimize its subgame utility **minus a KL penalty toward the imitation policy**. One knob λ moves the agent continuously between the behavioral-cloning anchor ($\lambda \to \infty$) and the unregularized equilibrium ($\lambda \to 0$), and in their Diplomacy experiments mild regularization *improves* strength rather than trading it away. Reproducing that tradeoff curve at small scale is this project's headline figure.

2 Setting: the 1-ply subgame

Let $\#$ be the counting measure.
Fix a game state of the standard ruleset in a movement phase; the search runs in no other phase (Sec. 9) and on no other ruleset. The reason for the second restriction is that the behavioral-cloning network, which supplies the candidates and the utilities (Secs. 5 and 6), has location tokens for the 81 locations of the standard map and for no other map.
Let n be the number of powers that have a unit in that state, numbered $1, \dots, n$; a power without a unit takes no part in the subgame.
Let an *order-set* of a power be one movement order for each of its units, all of them legal in the state.
For every power i, let its *candidate set* A_i be a finite, nonempty set of its order-sets; Sec. 5 says how it is drawn.
For every power i, let its *anchor* $\tau_i \in \Delta(A_i)$ be the behavioral-cloning policy restricted to A_i and renormalized (Sec. 5); $\tau_i(a) > 0$ for every $a \in A_i$.
Let a *joint profile* be a tuple $a = (a_1, \dots, a_n) \in A_1 \times \cdots \times A_n$, one candidate per power,

and for every power i let $u_i\colon A_1 \times \cdots \times A_n \to [0,1]$ be its *utility*: the predicted share of the final Sum-of-Squares score that power i holds after the joint profile is played (Sec. 6). These data define a one-shot n-player normal-form game, the *1-ply subgame*. piKL computes for every power a distribution over its candidates, its *mean strategy* (Sec. 3), and the agent plays its own orders from the mean strategy of its own power (Sec. 9).

Each power i conceptually maximizes the regularized utility (paper eq. 5)

$$U_i(\pi_i, \pi_{-i}) = u_i(\pi_i, \pi_{-i}) - \lambda_i \,\mathrm{KL}(\pi_i \,\|\, \tau_i), \qquad \lambda_i \in [0, \infty).$$

I also admit $\lambda_i = \infty$, the limit of Sec. 4 at which the agent is its anchor. The direction of the divergence is $\mathrm{KL}(\pi \,\|\, \tau)$. The paper chooses it for ease of analysis and implementation in regret minimization, and it is the behavior I want from an anchor: it forbids π from putting mass where τ has next to none (the paper's footnote 4 reports that the exact form of the penalty mattered less than doing any reasonable regularized search).

3 The anchored-hedge update (piKL-Hedge, paper Algorithm 1)

Let $N \geq 1$ be the number of iterations.
For every power i and every iteration $t \in \{0, \ldots, N\}$, let $\mathrm{CV}_i^t\colon A_i \to \mathbb{R}$ be the *cumulative action values* of power i after t iterations, with $\mathrm{CV}_i^0 \equiv 0$.
For every iteration $t \in \{1, \ldots, N\}$ let $\eta_t > 0$ be its *learning rate* (Sec. 7). The paper's Algorithm 1 states the update with one constant rate η, and its Appendix H sets η anew on every iteration; I write the rate of iteration t into the update as η_t.
Iteration t does three things, for every power at once.

1. **Policy.** For every power i, define its *iterate*

$$\pi_i^t(a) \propto \exp\left[\frac{\eta_t \mathrm{CV}_i^{t-1}(a) + t\lambda_i\eta_t \log \tau_i(a)}{1 + t\lambda_i\eta_t}\right], \qquad a \in A_i,$$

   computed as a softmax of the bracketed scores with their maximum subtracted first, so that the exponent stays finite.

2. **Play.** For every power i, draw $a_i^t \sim \pi_i^t$; all powers update simultaneously in self-play.

3. **Value update.** Every power i observes the others' sampled profile a_{-i}^t and updates **all** of its candidates:

$$\mathrm{CV}_i^t(a) = \mathrm{CV}_i^{t-1}(a) + u_i(a, a_{-i}^t) \qquad \text{for each } a \in A_i.$$

After N iterations the output of power i is its *mean strategy* (paper eq. 7)

$$\bar{\pi}_i^N = \frac{1}{N}\sum_{t=1}^{N} \pi_i^t,$$

and *not* the final iterate: the no-regret guarantees of Sec. 4 attach to the average. Beside the mean strategies the search reports, for every power, the iterate π_i^{N+1} that an iteration

$N+1$ would play from CV_i^N, the mean utilities CV_i^N/N, the number of iterations and the number of distinct joint profiles it evaluated (`PiklResult`).

Reading the exponent. Divide the numerator and the denominator by $t\eta_t$: the exponent is $(\mathrm{CV}_i^{t-1}/t + \lambda_i \log \tau_i)/(\lambda_i + 1/(t\eta_t))$. Since CV_i^{t-1}/t is, up to the factor $(t-1)/t$, the empirical mean utility $Q_i(a)$ of the candidate, the iterate is a softmax of $Q_i + \lambda_i \log \tau_i$ at inverse temperature $1/(\lambda_i + 1/(t\eta_t))$. As t grows the learning-rate term vanishes and the policy approaches the *anchored quantal response*

$$\pi_i(a) \propto \tau_i(a) \cdot \exp\Big(Q_i(a)/\lambda_i\Big),$$

the anchor reweighted exponentially by average performance, with λ_i as the temperature. With a uniform anchor this is a quantal response equilibrium (McKelvey & Palfrey 1995), which the paper notes as a special case; `anchored_quantal_response` computes it independently of the search, so that the validation of Sec. 8 can compare a converged mean strategy with a target it did not come from.

Update modes. Algorithm 1 as stated is the *sampled* update: one opponent profile is drawn per iteration, which is the only tractable choice when the joint space is large. In every iteration of sampled mode the search needs the utility of every one-deviation profile (a_{-i}^t, a), $a \in A_i$, of every power i. It asks the utility oracle of Sec. 6 for those of them it has not been given before, all in one batch with the repeated profiles removed, and it does not call the oracle when there are none. It keeps every utility it has ever been given in a cache for the whole search, so that a profile is evaluated once however often the iterates revisit it. The search stops with an error when the oracle returns a number of results different from the number of profiles asked, a result that does not give exactly one utility for every power, or a utility that is not finite.
I also implement *expected* updates, $\mathrm{CV}_i^t(a) \mathrel{+}= \mathbb{E}_{a_{-i} \sim \pi_{-i}^t} u_i(a, a_{-i})$, which enumerate the opponents' product distribution: the whole joint product $A_1 \times \cdots \times A_n$ is evaluated once up front, and every iteration feeds every power its exact expected utility vector. Expected updates are deterministic – no random draw enters – and have the same fixed points; their determinism makes the comparison of `run_pikl` with the self-play driver of `entente.gametheory` in Sec. 8 exact. They are exponential in the number of players, so the search refuses them beyond 200,000 joint profiles (`max_expected_profiles`), and Diplomacy always runs sampled. Sampled mode draws from a random generator the caller supplies (Sec. 9), or from `random.Random(0)` when the caller supplies none, so a run is reproducible from its seed.

4 Limits and guarantees

- $\lambda \to \infty$**.** The exponent tends to $\log \tau_i(a)$ for every $t \geq 1$, so $\pi_i^t = \tau_i$ exactly: the agent *is* the anchor. $\lambda = \infty$ is a legal configuration value (`.inf` in YAML) and the iterate is then the renormalized anchor itself, without any arithmetic; a finite λ so large that $t\lambda\eta_t$, or its product with the largest $|\log \tau_i(a)|$, overflows a double is treated the same way, since the mathematical limit is the same.
- $\lambda = 0$**.** The exponent is $\eta_t \mathrm{CV}_i^{t-1}(a)$: standard Hedge with learning rate η_t (the paper's eqs. 3–4 state Hedge on cumulative regrets, with no learning rate written; the softmax cancels the action-independent baseline, so cumulative values and cumulative regrets induce the same policy). The players' joint distribution of play

converges to a coarse correlated equilibrium in general, and the average policies converge to a Nash equilibrium in two-player zero-sum games.

- **KL bound (paper Theorem 1).** After N iterations in any multiplayer general-sum game, $\mathrm{KL}(\bar{\pi}_i^N \,\|\, \tau_i) \le (R_i^N/N + D_i)/\lambda_i$, where R_i^N is power i's regret with respect to the regularized utilities and D_i is any upper bound on the possible rewards of power i (Proposition 1 takes an upper bound on their range). With sublinear regret this tends to D_i/λ_i: **the distance to the anchor shrinks like** $1/\lambda$, which is the quantity my large-λ tests assert (Sec. 8).
- **Equilibrium bound (paper Theorem 2).** In a two-player zero-sum game every limit point of the average policies is, almost surely, a $\max_i(\lambda_i\beta_i)$-approximate Nash equilibrium of the *unregularized* game, where $\beta_i = \max_a \log(1/\tau_i(a))$ (paper eq. 21). So the exploitability is $O(\lambda)$ plus a finite-N regret term, which is the quantity my small-λ tests assert. A full-support anchor keeps β_i finite; the candidates are drawn from the behavioral-cloning policy and every anchor probability is kept strictly positive (Sec. 5), so this holds by construction.
- **Regret (paper Proposition 1).** With a constant $\eta \le 1/(\lambda_i\beta_i + 2D_i)$, the regret of piKL-Hedge with respect to the regularized utilities is $O(\log N)$:

$$R_i^N \le \frac{\log \#(A_i)}{\eta} + \frac{3e(1+\log N)}{\lambda_i \eta}\Big(D_i + \lambda_i\beta_i + \lambda_i\sqrt{\#(A_i)}\Big).$$

The scale of λ is tied to the scale of the payoffs, as the paper notes in its Appendix B: by eq. 5, doubling the utility range doubles the λ that produces the same anchoring. Every statement above is read, for Diplomacy, with my convention $u_i \in [0,1]$ of Sec. 2, so $D_i = 1$. The validation games of Sec. 8 keep their own payoffs, in $[-1,1]$ for matching pennies and in $[-2,2]$ for Kuhn poker, and the large-λ tests of these two games assert $\mathrm{KL}(\bar{\pi}_i^N \,\|\, \tau_i) < 2/\lambda_i$.

5 Candidate sampling (`candidates.py`)

The subgame is only as good as its candidate sets. For a power with at least one unit, `collect_candidates` builds its candidate set and its anchor in five steps.

1. **Draws.** It asks the behavioral-cloning policy for `attempts` order-sets in one call – by default $4k$, and never fewer than k. Each order-set is decoded autoregressively, one order per unit, at the configured `temperature` and nucleus `top_p` (the paper: temperature 0.5, top-p 0.95); where `include_greedy` is set, the first draw is the greedy (argmax) decode instead of a sample. Every draw is scored with its log-probability under the policy *at temperature 1*: temperature and nucleus shape the sampling only, never the anchor.
2. **Deduplication.** Two draws are the same order-set iff they hold the same orders; the first of them is kept with its score.
3. **Ranking.** The distinct order-sets are sorted by log-probability, descending, ties broken by their written text, and the first k are kept; fewer than k distinct order-sets is fine, since a forced position may not have k plausible ones. Where `include_`

`greedy` is set and the greedy decode is not among the k, it takes the place of the k-th, so that the greedy decode is in every subgame. The greedy decode gives every unit its most probable order given the orders decoded before it; it need not be the most probable order-set.

4. **Legality.** Every kept order-set is checked again: every order in it must be in `game.legal_orders`, and no two of its orders may be given at one location, so that no unit receives two orders. The check does not ask for an order for every unit; a unit that an order-set leaves without an order holds in the successor (Sec. 6). The decoder masks to `game.legal_orders` exactly (the masking-exactness test), so a failure here is a bug upstream, and the search stops with an error rather than searching from an illegal candidate.

5. **Anchor.** The anchor is the policy restricted to the kept set and renormalized, $\tau_i(a) = p_{\mathrm{BC}}(a) / \sum_{a' \in A_i} p_{\mathrm{BC}}(a')$, computed from the temperature-1 log-probabilities; a candidate whose log-probability is more than 700 below the best one's is given the best one's log-probability minus 700 instead, so that every $\tau_i(a)$ is strictly positive (Sec. 4). A kept order-set whose log-probability is not finite stops `collect_candidates` with an error.

Reweighting an anchor. `reweight_anchor` is the one seam through which outside information enters a subgame: it multiplies each candidate's anchor mass by a nonnegative *bias* the caller supplies and renormalizes. A bias of 1 everywhere changes nothing, a bias above 1 favors a candidate and one below 1 disfavors it; a bias of 0 reduces a candidate to a vanishing floor of its mass rather than to zero, so that the result is a strictly positive anchor whatever the bias, and a bias of 0 everywhere gives the anchor back unchanged. The bias is a vector of numbers and nothing else – never an order or an intent – so the search stays game-agnostic; the negotiation prototype translates a parsed message into a bias before it calls.

Paper scale: up to $N_c = 30$ candidates per power (≤ 3.5 per unit), 512 iterations. My plan scale: $k \sim$ 8–16 and $N \sim$ 64–256, smaller because my compute budget is a single local GPU.

6 1-ply utility estimation (`rollout.py`)

The utility of a joint profile is the value network's score of the successor state, with no deeper rollout: the paper's procedure (their Sec. 4.3.1, which follows Gray et al. 2021, with rollout length 0 by their Table 7).

Let the *successor* of a joint profile be the game obtained by submitting every power's order-set to a copy of the game and adjudicating one phase (`branch_game`); the engine's phase machine then passes every phase in which nobody has a decision, so the value network always sees a state where someone has one. That state is a retreat or an adjustment state whenever the adjudicated phase leaves some power such a decision. The value network was trained on movement-phase positions only, so it scores such a state outside its training distribution. A power absent from the profile holds with every unit. `branch_game` branches only a movement phase, the only kind of phase in which the search runs (Sec. 2); a retreat phase could not be branched in any case, since its adjudication reads the result

of the movement phase before it. A copy made for the search carries no history and is never serialized.

Let a *value function* be a function from a batch of games to one mapping from powers to scores per game (`value_fn`). In production it is the behavioral-cloning value head: for a game that is not over, the softmax of its 7-power Sum-of-Squares-share prediction, which lies in $[0, 1]$ and sums to 1 over the powers; for a game that is over, the exact Sum-of-Squares scores, since the truth is at hand and the network never saw a decided board in training (`value_scores`). The tests of `tests/search/test_rollout.py` pass an exact value function instead, `supply_share_value`, which scores every power by its share of the owned supply centers, and `rollout.py` imports no torch.

`profile_utility_fn(game, powers, candidates, value_fn)` builds the *utility oracle* of one position, the function `ProfileUtilityFn` that the search of Sec. 3 calls: given a batch of joint profiles, it branches the game once per profile, scores all the successors in **one** call of the value function – the batching seam a GPU needs – and returns, per profile, the utility of every power in the order of `powers`. The removal of repeated profiles and the cache of evaluated ones belong to the search, not to the oracle (Sec. 3): sampled play revisits profiles ever more often as the iterates concentrate, so the $O(N \sum_i \#(A_i))$ requests of a search become far fewer engine steps and network calls.

7 Hyperparameters and defaults

The agent's configuration is `PiklAgentConfig` (`search/config.py`): the checkpoint, the device, the fallback agent and the seed of Sec. 9, and under `search` the knobs of the search itself, `PiklSearchConfig`. Of the agent's own keys, `device` defaults to `cpu`, `fallback` to `greedy` and `seed` to 0, and `checkpoint` has no default. The table gives the keys of `PiklSearchConfig` and their defaults, against the paper's values.

Key	Meaning	Paper value	My default (`base.yaml`)
`k`	candidates per power	≤ 30 (≤ 3.5 per unit)	12
`attempts`	draws from the policy per power (Sec. 5); $4k$ when unset	–	48, that is $4k$ at $k = 12$
`iterations`	hedge iterations N	512	128
`lam`	anchor strength λ	$10^{-5} \ldots 10^{-1}$	1; the sweep sets $\{0, 0.3, 1, 3, \infty\}$
`eta.mode`	learning-rate schedule	adaptive	adaptive
`eta.scale`	the scale c of the adaptive schedule	10/3	10/3
`eta.value`	the rate of a constant schedule	–	unset; required in constant mode
`temperature`	candidate sampling temperature	0.5	0.5
`top_p`	nucleus cutoff	0.95	0.95
`include_greedy`	keep the argmax candidate	–	true
`play`	how the agent plays the mean strategy: `sample` or `argmax`	sample	sample

Expected updates (Sec. 3) are no configuration key: they are the `mode` argument of `run_pikl`, which the agent always calls in sampled mode and the tests in either.

The paper's λ range differs from my sweep grid. Their utilities and mine are both scaled to $[0, 1]$, but their anchor is a far stronger policy (an imitation-learning network trained on human games, their Appendix I), and at their $\lambda = 10^{-1}$ the search already predicts human orders as accurately as the anchor (their Sec. 4.3.2). My sweep (`lambda_sweep.yaml`) follows the grid $\{0, 0.3, 1, 3, \infty\}$, which spans KL bounds $D/\lambda \in \{\infty, 3.3, 1, 0.33, 0\}$ nats – wide enough to expose both regimes with a weak anchor; the table above gives the paper's decades for comparison, of which its main text reports 10^{-3} and 10^{-1}.

The η schedule. In adaptive mode (paper Appendix H), $\eta_t = c/(\sigma_t\sqrt{t})$ with c the scale and σ_t the population standard deviation, over the iterations $s < t$, of the expected utility $\langle \pi_i^s, u_i^s \rangle$ that the power's iterate earned in iteration s. The paper states the schedule in one sentence; this is the reading I take. While fewer than two iterations have run, or the spread is numerically zero (at most 10^{-12}; constant payoffs), σ_t is taken as 1, so that $\eta_t = c/\sqrt{t}$.
Constant mode (`eta.mode:constant`, `eta.value:v`) exists for the theory-faithful choice $\eta \leq 1/(\lambda_i\beta_i + 2D_i)$ of Sec. 4 and for the tests that compare the search with a learner of fixed rate (Sec. 8).

8 Validation before Diplomacy (`tests/search/`)

The validation makes quantitative assertions on games where everything can be computed exactly, in the spirit of the paper's Blotto illustration (their Appendix B), reusing `entente.gametheory`.

1. **Hand-computed iterates.** For one power with two candidates, fed a fixed stream of utility vectors, with a constant η and again with the adaptive schedule, the first iterates and the mean strategy are worked out by hand in the test and asserted to within 10^{-12} (`test_pikl_update.py`). This checks the arithmetic of the update: the $t\lambda\eta_t$ weight, the $1 + t\lambda\eta_t$ denominator, and that the iterate of round t reads CV^{t-1} and not CV^t.

2. **The two limits.** $\lambda = 0$ gives, iterate for iterate, the plain Hedge learner of `entente.gametheory.regret` to within 10^{-12}; $\lambda = \infty$ gives the anchor exactly at every iterate and as the mean, with a KL divergence of 0; a large finite λ tempers the first iterate toward the anchor as the formula says.

3. **Matrix games** (`test_pikl_matrix.py`, on `normal_form.py`: matching pennies and rock-paper-scissors with biased or uniform anchors, and for the quantal response a 2×2 zero-sum game with uniform anchors). The orchestrator `run_pikl` in expected mode agrees with the self-play driver of the gametheory package to within 10^{-12}, and sampled mode is deterministic given its seed. At λ near 0 the mean strategy of rock-paper-scissors is within 0.05 of uniform in every coordinate under expected updates, and the NashConv of matching pennies under sampled updates is below 0.12. At large λ the mean strategies of matching pennies stay within 0.02 of the anchors, and along $\lambda \in \{0.1, 0.3, 1, 3, 10\}$ the KL divergence of the mean strategy of rock-paper-scissors to its anchor does not increase by more than 10^{-6} from one λ to the next. The mean play at a fixed λ is within 0.02 of the anchored quantal response computed independently (Sec. 3).

4. **Kuhn poker** (`test_pikl_kuhn.py`, on `kuhn.py` and `exploitability.py`). Each player's pure strategies – the deterministic maps from information sets to actions – are the candidate sets, the exact expected payoffs over the six deals are the utility oracle, and the piKL mixture over pure strategies is converted to a behavioral strategy (Kuhn's theorem) and scored with the existing best-response exploitability; the reduction is checked first, the normal-form and the tree scores agreeing to within 10^{-9}. At λ near 0 the game value is within 0.03 of $-1/18$ and the exploitability is below 0.05; at large λ the mean strategy stays within 0.03 of a full-support anchor on every information set, within the bound of Theorem 1 (Sec. 4).

5. **The anchor seam.** `reweight_anchor` tilts and renormalizes as Sec. 5 says, is the identity for a bias of 1 everywhere, gives the anchor back for a bias of 0 everywhere, lets a growing boost dominate, and rejects a bias of the wrong length or a negative one (`test_pikl_reweight.py`).

Only with these green does piKL touch Diplomacy (`pikl_agent.py`), where no exact oracle exists.

9 Module map

Module	Role
`search/pikl.py`	Secs. 3–4: `AnchoredHedge`, one power's state and update; `run_pikl`, the simultaneous search of all powers over a utility oracle, with the cache and both update modes; `reweight_anchor`; `anchored_quantal_response`
`search/candidates.py`	Sec. 5: `collect_candidates` over a `CandidatePolicy` – draws, deduplication, ranking, legality, the anchor
`search/rollout.py`	Sec. 6: `branch_game`, `profile_utility_fn`, a torch-free supply-share value function for tests
`search/config.py`	Sec. 7: the configuration models of the agent, the search and the sweep, against which the committed YAML files are validated
`agents/pikl_agent.py`	the agent (below): the behavioral-cloning policy as `CandidatePolicy`, its value head as value function, the configuration applied
`scripts/run_lambda_sweep.py`	the sweep of Sec. 7: piKL in one seat against six anchor opponents, per λ, written as JSONL, from which the strength and agreement curves regenerate

The agent. `PiklAgent` plays a power as follows. In a phase that is not a movement phase it plays the `fallback` agent's orders, since the anchor policy is defined only where the behavioral-cloning policy is. In a movement phase it draws a candidate set for *every* power that has a unit (Sec. 5), builds the utility oracle of the position (Sec. 6), runs the search in sampled mode for `iterations` iterations (Sec. 3), and plays from the mean strategy of its own power: a draw from it when `play` is `sample`, its mode when `play` is `argmax`. Every power of the subgame is searched with the configured `lam` as its λ_i and the configured `eta` as its learning-rate schedule. Candidate sampling, the opponents' draws inside the search and the final draw share one random generator, seeded from the configured `seed`, the phase tag and the power, so that a decision replays exactly.

`search/pikl.py` is game-agnostic on purpose – players are indices and candidates are opaque – so the same code path is exercised by the exact-oracle tests of Sec. 8 and by Diplomacy, and the validation validates the production update rule.